Personal Reflection & Learning Journal: Cloud Mini Project

Student Name: Candidate CS22B001 | **Roll Number**: CS22B001 | **Course**: CS3081 (Cloud Computing)
GitHub Repository Link: https://github.com/assignment/cloud-project

1. Motivations and Initial Mental Models

When I first approached this project, my mental model of deploying Large Language Models was heavily influenced by managed cloud endpoints (e.g., OpenAI or Hugging Face Inference APIs). I viewed model serving primarily as an API integration task rather than a systems engineering problem. The assignment radically shifted my perspective. Distilling a 3.8-billion-parameter teacher (Phi-3.5) into an ultra-compact 70-million-parameter student (Pythia-70M) and running the entire inference lifecycle within strict hardware boundaries (1 vCPU, 1 GiB RAM) forced me to confront the gritty reality of compute limits, memory allocation, and OS-level thread contention.

I realized early on that deploying deep learning systems in production is not merely about model accuracy; it is fundamentally about *predictable resource consumption, fault tolerance, and deterministic operational guarantees*. A model that achieves high accuracy but exhibits erratic memory spikes or cold-start timeouts is unacceptable in production environments.

2. Deep Technical Learnings Across Project Milestones

2.1 Sequence-Level Knowledge Distillation on Constrained Hardware

My most significant conceptual learning was understanding why sequence-level knowledge distillation (Kim & Rush, 2016) succeeds where token-level or logit-based distillation fails under practical constraints. Initially, I assumed that matching logits via Kullback-Leibler divergence was inherently superior because it transfers continuous probability distributions over the vocabulary. However, I learned that vocabulary mismatches—specifically the disparity between Phi-3.5's 32,064-token tokenizer and Pythia's 50,304-token BPE tokenizer—make direct logit comparison mathematically intractable without introducing complex auxiliary linear projection layers.

Furthermore, sequence-level distillation drastically simplified the data ingestion pipeline: the teacher acts as an autonomous data annotator producing supervised targets. Training Pythia-70M on 1,500 teacher-annotated examples transformed the base model from producing total syntactic gibberish (0% JSON validity) to achieving 99.0% JSON validity and 46.0% full-frame accuracy, confirming that small models have sufficient inductive capacity to learn structured domain schemas when supervised by high-quality teacher sequences.

2.2 Cloud-Native Decoupling: Object Storage vs. Container Images

A critical cloud design pattern emphasized in this project was the strict decoupling of compute containers from static model artifacts. Prior to this assignment, my default instinct was to bundle model weights directly into the Docker image via COPY models/ /app/models/. I quickly recognized the severe operational flaws of that approach: bloating the container image to several gigabytes, defeating Docker layer caching, slowing image pull times across cluster nodes, and tightly coupling model retraining with container re-builds.

By hosting model weights in SeaweedFS S3-compatible buckets and dynamically pulling them at container startup, the container image remained extremely compact (1.28 GB using CPU-only wheels). This architectural separation enabled independent model versioning (v1 vs. v2) and seamless Kubernetes rolling updates without requiring container image rebuilds.

3. Major Challenges Faced and How They Were Overcome

3.1 The CPU Multi-Threading Bottleneck and Thread Contention

Challenge: During initial PyTorch training benchmarks on my CPU laptop, execution was agonizingly slow—a single batch of 8 samples required upwards of 44 seconds. CPU utilization was pinned at 100% across all 16 logical cores, yet backward pass throughput was near zero.

Diagnosis & Resolution: Profiling revealed severe lock contention within the Windows OpenMP thread pool. PyTorch's default behavior spawned 16 threads that continually contended for thread synchronization barriers on small matrix operations. By explicitly setting torch.set_num_threads(2) before initializing the model, thread switching overhead was eliminated. Batch execution latency immediately dropped from 44.0 seconds to 0.77 seconds (a 57x speedup). This taught me that more CPU cores do not equate to faster compute when synchronization overhead dominates.

3.2 Half-Precision Numerical Instability on CPU Architectures

Challenge: After completing one training batch, subsequent iterations immediately produced loss = nan and corrupted the model weights.

Diagnosis & Resolution: Pythia-70M weights are stored by default in torch.float16. While modern GPUs provide native hardware acceleration and gradient scalers for FP16, x86 CPU FP16 emulation lacks dynamic loss scaling. Gradients computed during AdamW optimization experienced severe underflow and numerical overflow, resulting in NaN values. Calling model = model.float() converted the weights to float32, completely eliminating NaN instability and enabling smooth, monotonic loss convergence (0.1788 $\to$ 0.0704 $\to$ 0.0491).

3.3 Subword Token Boundary Merging and Label Alignment

Challenge: During early fine-tuning, the student model generated empty strings or predicted end-of-text tokens immediately after the prompt.

Diagnosis & Resolution: I discovered that BPE tokenizers greedily merge whitespace with subsequent punctuation. When formatting prompts as Command: ...\nJSON: , the trailing space merged with the opening curly brace {" into a unified token ■{". Tokenizing the prompt prefix separately resulted in length mismatches that caused the training mask to inadvertently mask out the opening brace of the target JSON. Replacing space delimiters with explicit newlines (\nJSON:\n) established rigid token boundaries, ensuring exact label alignment.

4. Networking, Subnetting, and Stateless Security Insights

Working through Task 5 gave me a profound appreciation for low-level networking. In academic coursework, subnetting often feels like abstract binary arithmetic. Calculating contiguous VLSM subnets for 10.20.0.0/16 (allocating /22 for 1,000 app hosts, /24 for public, /25 for data, and /27 for management) grounded those concepts in concrete infrastructure topology.

More importantly, configuring the Network Access Control List (NACL) reinforced the fundamental distinction between **stateful security groups** and **stateless firewalls**. Realizing that a stateless NACL has zero connection tracking—meaning an allowed inbound HTTP request on port 80 cannot return a response unless an explicit outbound rule permits traffic to the client's ephemeral port range (1024–65535)—was a major revelation. Furthermore, proving that rule evaluation order matters (placing the deny rule for abusive CIDR 198.51.100.0/24 before web allow rules) underscored how subtle misconfigurations create catastrophic security vulnerabilities.

5. Critical Observations, Findings, and Empirical Insights

5.1 Performance, Reliability, and Cost Trade-offs

The Locust load test provided tangible operational benchmarks: sustaining 8.33 requests/second with a median latency of 110 ms and a 95th percentile latency of 245 ms under 4 concurrent users. The measured error rate was 0.2% (comfortably meeting the 1.0% limit), consuming only 20% of our defined error budget.

Translating these metrics into cloud economics using the hypothetical rate card (■3.50/vCPU-hr, ■0.45/GiB-hr) revealed an estimated cost of **■0.26 per 1,000 requests**. This highlights the compelling cost advantage of distilled edge models: running specialized 70M parameter models on commodity CPU compute is orders of magnitude cheaper than paying per-token charges to third-party proprietary LLM APIs.

5.2 Microservice Lifecycle and Probe Dynamics

Observing Kubernetes probe dynamics in real time was deeply instructive. Measuring the 14.1-second cold-start latency (8.4s for S3 weight retrieval, 5.7s for in-memory model instantiation) demonstrated why readinessProbe configurations are non-negotiable. Without a readiness gate on /readyz, Traefik routed incoming requests to uninitialized Pods, causing immediate connection resets. With probes configured, zero downtime was achieved during rolling updates from v1 to v2.

6. What Could Have Been Done Better & Future Trajectory

Reflecting on the final architecture, several clear engineering enhancements could be pursued in future iterations:

1. **Model Quantization & Runtime Engines**: Serving the model via raw PyTorch in Python incurs significant interpreter overhead. Exporting Pythia-70M to ONNX Runtime or llama.cpp with INT8/INT4 quantization would slash RAM consumption from 618 MiB to under 150 MiB and double token generation throughput on CPU.
2. **Dynamic Weight Caching & Shared Volumes**: Rather than downloading model weights from S3 on every fresh pod spin-up (which accounts for 60% of cold-start latency), deploying a shared ReadWriteMany persistent cache volume across cluster nodes would reduce cold-start time from 14.1s to under 5 seconds.
3. **Multi-Node Cluster Topology**: Testing on a single-node k3d cluster validated software controllers but could not simulate true distributed system failure domains (such as node network partitions or physical hardware loss). Validating across a multi-node cluster would provide stronger production fidelity.
4. **Dynamic Secret Management**: Replacing static Kubernetes Secret credentials with dynamic workload identity (e.g., HashiCorp Vault or AWS IAM Roles for Service Accounts) would eliminate hardcoded credential exposure.

7. Concluding Remarks

This project provided an invaluable synthesis of machine learning theory, Linux systems programming, containerization, cloud networking, and distributed systems reliability. Navigating hardware bottlenecks, debugging PyTorch thread contention, implementing stateless packet filtering, and writing automated CI/CD quality gates taught me lessons that no textbook alone could convey. I emerge with a far deeper, more pragmatic understanding of how to build, deploy, and scale robust cloud-native AI systems.

Candidate Declaration: I hereby confirm that this reflection document, codebase, and accompanying academic report represent my individual, verified work for the CS3081 Cloud Computing Mini Project.

Signature: *Candidate CS22B001* **Date**: November 6, 2026